



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de Hermosillo
Ingeniería en Sistemas Computacionales
Programación Web

ProyectoFinal-PuntoDeVenta

Profesor:

Jonathan Adrian Moroyoqui Ibarra

Alumnos S6B:

Rommel Eduardo de La Re Lopez // 23330497

Jorge Luis Moreno Ayala //23330556

Hermosillo, Sonora, 28 de Mayo de 2026

Introducción

Descripción general del sistema desarrollado

El Sistema POS (Point of Sale) es una aplicación web desarrollada con ASP.NET Core Web API (.NET 10) en el backend y React con Vite en el frontend. El sistema permite gestionar las operaciones básicas de un punto de venta: administración de productos, categorías y clientes, así como el registro y seguimiento de ventas en tiempo real.

La aplicación sigue una arquitectura cliente-servidor desacoplada, donde el backend expone una API RESTful consumida por el frontend a través de peticiones HTTP. La base de datos es administrada mediante Entity Framework Core con el enfoque Code First, lo que permite generar y versionar el esquema de la base de datos directamente desde el código.

Problema que resuelve

Muchos negocios pequeños y locales operan sus ventas de manera manual o con hojas de cálculo, lo que genera errores en el control de inventario, dificultad para rastrear el historial de ventas y falta de visibilidad sobre el desempeño del negocio. Este sistema centraliza la información de productos, clientes y transacciones en una sola plataforma accesible desde el navegador, reduciendo errores y mejorando la eficiencia operativa.

Objetivo general del proyecto

Desarrollar un sistema web de punto de venta que permita gestionar el catálogo de productos, clientes y categorías, así como registrar ventas, mediante una interfaz intuitiva construida en React que se comunica con una API RESTful desarrollada en ASP.NET Core, utilizando Entity Framework Core Code First para la persistencia de datos en SQL Server.

Documentación Técnica

Tecnologías utilizadas

Backend

Tecnología / Librería	Versión	Uso
.NET	10.0	Framework base del backend
ASP.NET Core Web API	10.0	Exposición de endpoints REST
Entity Framework Core	9.x / 10.x	ORM para acceso a base de datos
EF Core SqlServer	Compatible .NET 10	Proveedor SQL Server para EF Core
EF Core Tools / Design	Compatible .NET 10	Migraciones y scaffolding
Scalar.AspNetCore	Última estable	UI de documentación del API

Frontend

Librería	Versión	Uso
React	19.x	Librería principal de UI
Vite	8.x	Bundler y servidor de desarrollo
react-router-dom	7.x	Navegación entre vistas (SPA)

Base de datos

Componente	Detalle
SQL Server	2019 o superior (local o Express)
Nombre de la base de datos	SistemaPOSDB
Generación del esquema	Migraciones EF Core (dotnet ef database update)

Arquitectura

Carpeta / Archivo	Contenido
Controllers/	Controladores de la API REST (un archivo por entidad)
Models/	Entidades del dominio: Categoria, Producto, Cliente, Venta, DetalleVenta
Data/	AppDbContext.cs — configuración de EF Core y DbSet
Dtos/	Objetos de transferencia de datos (entrada y salida de los endpoints)
Migrations/	Migraciones generadas por EF Core, versionadas en el repositorio
vista/	Proyecto React + Vite (frontend completo dentro del repo .NET)
vista/src/pages/	Vistas: Dashboard, Productos, Categorías, Clientes, NuevaVenta
vista/src/services/	api.js — funciones centralizadas de fetch al backend
vista/src/components/	Componentes reutilizables: Navbar, modales, tablas

Base de datos

Entidad	Atributo	Tipo	Descripción
Categoria	Id	int (PK)	Identificador único
	Nombre	string	Nombre de la categoría (req.)
	Descripcion	string?	Descripción opcional
Producto	Id	int (PK)	Identificador único
	Nombre	string	Nombre del producto (req.)
	Precio	decimal	Precio unitario
	Stock	int	Unidades disponibles
	CategoriaId	int (FK)	Referencia a Categoria
Cliente	Id	int (PK)	Identificador único
	Nombre	string	Nombre completo (req.)

	Telefono	string?	Teléfono opcional
	Email	string?	Correo electrónico opcional
Venta	Id	int (PK)	Identificador único
	Clienteld	int (FK)	Referencia a Cliente
	Fecha	DateTime	Fecha y hora de la venta
	Total	decimal	Monto total calculado
DetalleVenta	Id	int (PK)	Identificador único
	Ventald	int (FK)	Referencia a Venta
	Productold	int (FK)	Referencia a Producto
	Cantidad	int	Unidades vendidas
	PrecioUnitario	decimal	Precio al momento de la venta

Categoria → Producto: una categoría puede tener muchos productos (1:N)

Cliente → Venta: un cliente puede tener muchas ventas (1:N)

Venta → DetalleVenta: una venta puede tener muchos detalles (1:N)

Producto → DetalleVenta: un producto puede aparecer en muchos detalles (1:N)

Api

Método	Ruta	Descripción
GET	/api/categorias	Obtener todas las categorías
GET	/api/categorias/{id}	Obtener una categoría por ID
POST	/api/categorias	Crear nueva categoría
PUT	/api/categorias/{id}	Actualizar categoría existente
DELETE	/api/categorias/{id}	Eliminar categoría por ID
GET	/api/productos	Obtener todos los productos
GET	/api/productos/{id}	Obtener un producto por ID
POST	/api/productos	Crear nuevo producto
PUT	/api/productos/{id}	Actualizar producto existente
DELETE	/api/productos/{id}	Eliminar producto por ID
GET	/api/clientes	Obtener todos los clientes
GET	/api/clientes/{id}	Obtener un cliente por ID
POST	/api/clientes	Crear nuevo cliente
PUT	/api/clientes/{id}	Actualizar cliente existente
DELETE	/api/clientes/{id}	Eliminar cliente por ID
GET	/api/ventas	Obtener todas las ventas con detalles
GET	/api/ventas/{id}	Obtener una venta con sus detalles
POST	/api/ventas	Registrar nueva venta (calcula total automáticamente)

Distribución del trabajo

Backend

Integrante	Tareas desarrolladas en el backend
Rommel De La Re	- Setup inicial del proyecto .NET 10 y estructura de carpetas. - Configuración de la API con Scalar y políticas de CORS. - Controladores y DTOs para Categorías y Productos. - Implementación completa del módulo de Ventas (Modelos de Venta y DetalleVenta, DTOs, VentasController y migraciones de ventas). - Configuración de Seed Data (datos de prueba) en la base de datos.
Jorge Moreno	- Creación de los modelos base (Categoria, Producto, Cliente) y configuración inicial del ApplicationDbContext. - Generación de la migración inicial de EF Core (InitialCreate) y creación de la base de datos. - Implementación del controlador y DTOs para Clientes (ClientesController).

Frontend

Integrante	Tareas desarrolladas en el frontend
Rommel De La Re	- Setup inicial del proyecto Vite + React en la carpeta "/vista". - Configuración base de React Router, componente Navbar y archivo centralizado "api.js". - Desarrollo completo de la vista del Dashboard.
Jorge Moreno	- Desarrollo de las vistas de Categorías y Productos con sus formularios CRUD completos. - Desarrollo de la vista de Clientes con formularios CRUD. - Desarrollo de la vista de Nueva Venta (incluyendo el flujo de creación de venta, selector de cliente y productos).

Retroalimentación del proyecto

Rommel De La Re

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

Errores en la api, lo resolvimos con codex.

¿Qué herramientas o recursos externos consultaron?

Codex.

¿Qué cambiarían o mejorarían si tuvieran más tiempo?

Cambiaría el frontend y agregaría más cosas al dashboard.

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

git.

Jorge Moreno

¿Qué fue lo más difícil de desarrollar y cómo lo resolvieron?

Los endpoints, lo resolvimos con codex.

¿Qué herramientas o recursos externos consultaron?

Codex.

¿Qué cambiarían o mejorarían si tuvieran más tiempo?

El frontend.

¿Qué aprendieron que no sabían antes de iniciar el proyecto?

La estructura de un proyecto fullstack y git.

Retroalimentación del curso

Rommel De La Re

¿Qué temas del curso fueron más útiles para el proyecto?

React, vite

¿Qué temas necesitan más práctica o explicación?

.NET, las migraciones.

¿Qué le agregarían o quitarían al curso?

Le agregaría una explicación más detallada a que hace cada cosa.

Calificación del curso (1-10) y justificación

9, Siento que fue muy apresurado el backend.

Comentarios adicionales para el profesor

Algo bien.

Jorge Moreno

¿Qué temas del curso fueron más útiles para el proyecto?

Todo lo relacionado al frontend.

¿Qué temas necesitan más práctica o explicación?

Todo lo relacionado al backend.

¿Qué le agregarían o quitarían al curso?

Explicaciones a fondo de lo que se está haciendo.

Calificación del curso (1-10) y justificación

8, Me hubiera gustado ver con más calma la programación en general.

Comentarios adicionales para el profesor

Muy buena onda.

Conclusiones

Conclusión general del equipo

El desarrollo del Sistema POS permitió al equipo integrar los conocimientos adquiridos durante el curso en un proyecto funcional y completo. La combinación de ASP.NET Core Web API con React demostró ser una arquitectura sólida y escalable para aplicaciones web modernas, donde el desacoplamiento entre frontend y backend facilita el mantenimiento y la evolución independiente de cada capa.

¿Se logró el objetivo planteado?

Sí. El sistema cumple con los requisitos establecidos al inicio del proyecto: tres entidades con CRUD completo (Categorías, Productos, Clientes), dos módulos funcionales (Ventas e Inventario), cinco vistas en React con navegación mediante React Router, migraciones de EF Core versionadas en el repositorio y documentación del API disponible mediante Scalar en modo desarrollo.

Reflexión final sobre el trabajo en equipo

El trabajo en equipo durante este proyecto fue muy ameno ya que teníamos buena comunicación durante todo el desarrollo.